

The Reactive Sandbox - Companion Doc

AI 201 — CONTEXT & FRAMEWORK

The Reactive Sandbox

Architecture, domain guidance, ESF practices, and the Art Director framing for Project 2.

This document is the companion to the Project 2 Brief. The Brief tells you what to build and when it's due. This document tells you *why* and *how*.

The Architecture: Browser → Detail → Controller

Every domain in this assignment shares the same architectural spine. Three UI components read from and write to a single shared state object. That's it. The entire assignment is about learning to manage that connection.

The Browser lets the user explore a collection. It reads from shared state to know what's available and writes to shared state when the user makes a selection.

The Detail View displays rich information about the currently selected item. It only reads from shared state — it reacts to changes, it doesn't initiate them.

The Controller lets the user take actions that modify the system — filtering, assigning, toggling, comparing. It reads and writes. When the Controller changes state, both the Browser and Detail View respond.

■ **THE ARCHITECTURAL RULE:** No component manages its own copy of the data. State lives in one place. Components receive it as props and send changes back up. If you find yourself storing the same data in two places, stop. That's the bug this project teaches you to avoid.

React Concepts You're Learning

This project is a direct application of the React documentation's *Sharing State Between Components* pattern. The key concepts:

Lifting State Up: When two or more components need the same data, you move that data to their nearest common parent. The parent holds the state; children receive it as props. This is the single most important React pattern you will learn in this course.

Props Down, Events Up: Data flows downward through props. User actions flow upward through callback functions. The parent decides what changes; the children just report what happened.

Single Source of Truth: For every piece of data in your app, exactly one component “owns” it. Every other component that needs it gets a copy via props. This prevents contradictions and makes your system predictable.

JSON as Your Data Layer: Your shared state is a JSON object. Your Browser maps over arrays in that object. Your Detail View reads properties from the selected item. Your Controller modifies values in the state. JSON is the language your components speak.

Domain Deep Dives

The assignment is domain-agnostic. Below are detailed breakdowns of how the Browser → Detail → Controller pattern applies to specific fields. Use these as starting points, not prescriptions.

Game UI

The original framing from the course's game UI scenario. A spellbook, inventory terminal, or deck builder where the interface feels like a physical object inside the game world. This is *diegetic design* — the UI exists within the fiction. Dead Space's health bar on the character's spine. The Pip-Boy in Fallout. The holographic displays in Metroid Prime.

Shared state example: { selectedCard: “fireball”, deck: [...], collection: [...], filterBy: “offensive” }

Reference: react.dev/learn/sharing-state-between-components

Contracting / Trades

A job management system for a contracting business. The Browser shows active jobs filtered by status. The Detail View shows a job's scope, materials, estimate, and client info. The Controller handles crew assignment, scheduling, and status changes.

Shared state example: { selectedJob: "job-042", jobs: [...], crews: [...], filterStatus: "in-progress" }

This is not a toy. This is a tool someone could actually use. The student who builds this well leaves the class with a portfolio piece *and* a working prototype for a family business.

Project Management

Every PM tool you've used — Jira, Asana, Linear, Monday — is this exact three-panel pattern. Task board as Browser, task detail as Detail View, status/priority/assignee controls as Controller. You're not copying these tools. You're understanding the architecture underneath them.

Shared state example: { selectedTask: "task-17", tasks: [...], team: [...], view: "kanban" }

Sports Media

ESPN, The Athletic, every fantasy sports app — player roster as Browser, player profile and stats as Detail View, comparison and filtering tools as Controller. The data model is naturally rich: positions, stats, seasons, matchups.

Shared state example: { selectedPlayer: "p-23", roster: [...], compareList: [...], statCategory: "season" }

Portfolio / UX

For UX and interaction design students: your portfolio site is not a static page — it's a system that responds to interaction. Project gallery as Browser, case study as Detail View, filter and sort controls as Controller. You understand every piece of it because you built the state machine.

Shared state example: { selectedProject: "proj-5", projects: [...], filterTags: ["UX"], viewMode: "grid" }

The Art Director Framing

In Project 1, you were the Art Director of a single screen. In Project 2, you are the Art Director of a **system**. The difference matters.

A single screen is a poster. A system is a machine. The Art Director of a machine doesn't just decide how it looks — they decide how it *behaves*. What happens when you click? What changes? What stays the same? What's the rhythm of the interaction?

Your Design Intent for this project is therefore more architectural than visual. Yes, you still define palette, typography, and mood. But you also define the data model, the state flow, and the interaction rules. You are writing the spec for a machine, not a painting.

Your Design Intent should answer these questions before AI touches code:

- What is my domain and why?
- What data does my system manage? (Sketch the JSON shape.)
- What are the three panels and what does each one do?
- When the user clicks in the Browser, what happens in the Detail View?
- When the user takes action in the Controller, what changes in the Browser?
- What is the visual mood? (Palette, type, spacing.)
- What should it feel like to use this system?

ESF Practices for Project 2

Design Intent

Same practice as P1, higher stakes. Your Design Intent now includes architectural decisions (data model, state flow, panel responsibilities) alongside visual decisions (palette, typography, mood). Write it before Session 8. This is the document against which you evaluate every line of AI-generated code.

AI Direction Log

3–5 entries documenting your editorial relationship with AI across the project. Focus on the decisions that shaped the system. Good log entries sound like: “I asked Claude to wire the Browser selection to the Detail View via lifted state. It used useContext instead of props. I reverted to props because the component tree is only two levels deep and Context would be overengineering.”

Records of Resistance

Three documented moments where you rejected or significantly revised AI output. In a three-panel system, the AI will almost certainly try to duplicate state across components, add unnecessary complexity, or over-style before the architecture is solid. Those are your resistance moments. Document them.

■ COMMON AI FAILURE MODES FOR THIS PROJECT:

1. Duplicating state — AI stores the selected item in both the Browser and the Detail View instead of lifting it.
2. Over-scoping — AI adds features you didn’t ask for (routing, authentication, database calls).
3. Styling before structure — AI produces beautiful CSS before the state management works.
4. useContext/Redux too early — AI reaches for advanced state management when useState + props is sufficient.

These are not mistakes. They are opportunities. Document them.

Five Questions

Before you submit, answer these in a short paragraph. They go in your README or sketchbook:

- 1. Can I defend this? Can I explain every major decision — especially the state architecture?
- 2. Is this mine? Does this reflect my creative direction, or did I mostly follow AI's direction?
- 3. Did I verify? Do the three panels actually share state, or are they faking it?
- 4. Would I teach this? Could I explain the props-down/events-up pattern to a classmate?
- 5. Is my documentation honest? Does my AI Direction Log accurately describe what I asked and what I changed?

Tips for Working with AI on This Project

- 1. Build the state model first.** Before you ask AI to write any components, write the JSON shape of your state by hand in your Design Intent. When AI knows the data model, its code is dramatically better.
- 2. One component at a time.** Prompt AI to build the Browser first. Get it rendering from mock data. Then the Detail View. Then the Controller. Then wire them together. Don't ask AI to build all three at once — it will make architectural decisions you didn't approve.
- 3. Test the wiring before the styling.** Click something in the Browser. Does the Detail View update? Change something in the Controller. Does the Browser reflect it? If yes, then add visual polish. If no, the architecture is broken and no amount of CSS will fix it.
- 4. Commit before and after AI sessions.** Your Git history is your undo button. If AI breaks the state management, you can revert to the last working version. This is the Cloud Save from Session 4 — same skill, higher stakes.
- 5. Stay in your current pass.** The production pipeline from P1 applies here: structure → state → data → interaction → polish. If you're in the state pass, don't let AI distract you with animations.

What's New Since Project 1

	Project 1	Project 2
Scope	One screen	Three connected panels

State	Hover states (CSS only)	Centralized React state (useState, props)
Data	Hardcoded content	JSON data model
Components	HTML + CSS Grid	React components with props
Interaction	Hover effects	Click events, state changes, cross-panel updates
Design Intent	Visual spec	Visual + architectural spec
Your Role	Art Director of a poster	Art Director of a system

ESF DIRECTIVE MEMO This section authored by Professor Tim Lindsey without AI assistance, per SCAD ESF Protocol.

Designing systems is what we do in GAME. Single-source-of-truth data is what our architects are architecting. In this assignment, you will begin building an app that pulls data from a single place to adjust the look, functionality, and usability of three different systems. In games, this is a state machine, but the concept is universal. Playing “the telephone game” with your data, copying it, hosting it in different places at different times, and using temporary variables are all bad habits that neither save time nor produce trustworthy results. This assignment teaches that.

In this assignment, you are going from designing where the user's eyes and clicks go to understanding how you use the DATA to drive the USER to good outcomes.

DISCLOSURE: Document created with AI assistance (Claude, Anthropic) for formatting, structure, and domain examples. All pedagogical decisions, content, assessment criteria, and curriculum design directed by Professor Tim Lindsey. Directive Memo authored without AI, per SCAD ESF Protocol.

v1.0 | April 2026 | AI 201 Spring 2026